未来智能无人艇集群
协同巡航海域

基于 Three.js 的大型三维海洋场景
开发文档



作者： 朱华天

模型： DeepSeek V4.1 Flash 1017

日期： 2026 年 10 月 21 日

项目仓库： https://github.com/ZHT1235456/OceanSwarm

版本 v1.0 | Three.js 0.186 | Vite 7 | 全部场景实时程序化生成

摘要

本文档完整记录“OceanSwarm ——未来智能无人艇集群协同巡航海域”这一大型三维海洋场景的设计与实现。项目使用 Three.js 与 Vite 构建，在不依赖任何外部模型、贴图与 HDR 资源的前提下，以纯程序化方式生成一片数公里尺度的开放海域：17 艘不同任务的智能无人水面艇（含一艘 30 米三体无人指挥旗舰）沿宽阔圆弧航线协同巡航，海面上分布着连续的历史轨迹尾迹、破浪泡沫与克制的青蓝色通信光效；远景由岩石海岛、海上无人系统试验基地、保障母船、珊瑚礁潟湖、航标灯塔与风电场共同构成层次分明的海天空间。文档依次介绍项目目标、运行方式、总体架构、程序化海洋与天空、无人艇建模、编队动力学、尾迹与光效、海域环境、美术与光照方案、性能与质量验证，并给出完整的参数表、代码摘录与实机截图。项目可直接通过 `npm run dev` 运行，构建产物约 860 KB（gzip 242 KB），实测约 549 个绘制调用、78 万个三角形，控制台零错误、界面零 UI。

关键词： Three.js；Vite；无人水面艇（USV）；程序化建模；海洋着色器；三渲二；实时渲染

目录

1 项目概述 **1**
1.1 背景与目标 . . . 1
1.2 最终成果 . . . 1
1.3 需求与实现对照 . . . 2

2 运行与操作 **3**
2.1 环境与命令 . . . 3
2.2 交互方式 . . . 3
2.3 开发用取景预设 . . . 3

3 总体架构 **4**
3.1 模块划分 . . . 4
3.2 帧循环与数据流 . . . 5
3.3 代码结构与规模 . . . 5
3.4 全局约定 . . . 6

4 程序化海洋 **6**
4.1 七组方向波 . . . 6
4.2 CPU 与 GPU 的一致性 . . . 6
4.3 海面网格：径向加密与相机跟随 . . . 7
4.4 片元着色 . . . 7

5 天空、大气与光照 **9**
5.1 天空穹顶 . . . 9
5.2 海天无缝：共享天空色函数 . . . 9
5.3 云与太阳 . . . 10
5.4 光照与材质响应 . . . 10

6 无人艇程序化建模 **11**
6.1 统一设计语言与尺度 . . . 11
6.2 参数化船体生成 . . . 11
6.3 Builder：合批、描边与动态部件 . . . 12
6.4 旗舰设计 . . . 12
6.5 材质库与三渲二外观 . . . 13

7 编队动力学与航行 **13**
7.1 航线 . . . 13
7.2 阵位与自然摆动 . . . 14
7.3 跟随与航向控制 . . . 14
7.4 随浪姿态 . . . 14

8 尾迹、泡沫与协同光效 **15**
8.1 历史轨迹尾迹 15
8.2 泡沫着色器 15
8.3 协同可视化光效 16
8.4 海鸥 16

9 海域环境构建 **16**
9.1 海岛地形 16
9.2 珊瑚礁潟湖 17
9.3 海上试验基地 17
9.4 母船与航标设施 18

10 美术方向、色彩与构图 **19**
10.1 三渲二的混合管线 19
10.2 主色板 19
10.3 构图与镜头语言 19

11 性能优化与质量验证 **20**
11.1 性能预算与优化手段 20
11.2 实测数据 20
11.3 自动化视觉验证 21
11.4 已知限制与后续方向 21

12 总结 **22**

A 波场采样与姿态计算补充 **22**

B 项目目录结构 **22**

1 项目概述

1.1 背景与目标

本项目以“未来海洋无人系统试验海域”为主题，目标是制作一个可以直接在浏览器中自由探索的大型三维场景：观众打开页面后，应立刻看到辽阔连续的蓝色海洋、清晰的海天交界线、一支正在协同巡航的无人艇集群，以及由海岛、海上基地、珊瑚礁与远方云层构成的完整海洋世界。

与常见的游戏 Demo 不同，本项目在约束上非常明确：

- **无界面**：画面中不允许出现任何 UI 面板、按钮、文字说明、坐标轴、调试信息或浏览器界面元素，打开即是完整场景；
- **无外部资源**：所有船体、地形、天空、云、水面、泡沫与光效均由程序化几何与着色器实时生成，不使用外部模型、贴图或 HDR 环境图；
- **真实尺度感**：以米为单位建模，海域半径约 26 km，无人艇 6–30 m，海岛 300–1250 m，海上指挥塔约 62 m，通过近大远小、空气透视与雾效形成强烈纵深感；
- **优先保证大效果**：在性能预算内优先保证海洋连续性、编队完整度、旗舰精细度、尾迹与海天氛围，其次才是细小的装饰。

1.2 最终成果

最终场景包含：

- 一片半径 26 km 的程序化海面，CPU/GPU 共用同一套 7 组方向波，海面网格随相机移动，与天空穹顶共享天空色函数，地平线无接缝；
- 17 艘无人艇：1 艘 30 m 三体旗舰、5 艘高速侦察艇、3 艘环境监测艇、2 艘双体测绘艇、2 艘多用途巡逻艇，以及通信中继、能源保障各 1–2 艘和 1 艘无人救援艇；
- 完整的编队行为：旗舰沿半径约 1.6 km 的大圆弧缓慢转向，各艇以指数弹簧保持阵位并带自然摆动，侦察艇周期性向外展开，所有船体随浪升沉、纵摇与横摇；
- 每艘船独立的轨迹尾迹、船首 V 形破浪、船侧泡沫线、尾流泡沫，共 17 套动态泡沫网格；
- 左后方海岛上的无人系统试验基地（62 m 指挥塔、模块化科研建筑、封闭连廊、三组桩基码头、集装箱场、防波堤）、120 m 保障母船；右前方浅海珊瑚礁潟湖（海底沙脊、498 组珊瑚、海草、三个鱼群、焦散光斑）；中远景的三座海岛、航标浮标阵列、灯塔、自升式观测平台与五台海上风机；
- 程序化天空穹顶、三层云（积云精灵、高空卷云、地平线薄雾）、低角度暖阳、太阳光晕与海面碎金反射；少量海鸥与克制的青蓝色协同可视化光效。



图 1　初始镜头下的完整场景：连续海面、无人艇编队、白色尾迹、左侧研究海岛与远方海天交界线。

1.3　需求与实现对照

表 1 概括了需求要点与对应的实现手段。

表 1　核心需求与实现方式对照

需求要点	实现方式
无 UI、打开即场景	全屏 canvas，无任何 DOM 覆盖层；相机与场景由 `src/main.js` 直接装配
广阔连续海洋	26 km 半径径向网格 + 相机跟随 + 距离波衰减 + 方向相关雾色
清晰海天交界线	海面雾色调用天空穹顶同一 `skyTint()` 函数，逐方向无缝融合
15–20 艘集群	17 艘：1 旗舰 + 16 艘 7 个类别，程序化参数化船体与模块化设备
精细中央旗舰	30 m 三体结构、13 个材质批次合批 + 描边、3 组动态设备、61k 三角形
白色航行尾迹	每船历史轨迹带 + 船首 V 形臂 + 泡沫项圈 + 尾流，自定义泡沫着色器
岛屿与海上基地	噪声高度场海岛、分层岩壁、程序化建筑群、桩基码头与地形高度采样贴合
天空阳光云层	程序化穹顶 + 画布积云 + 卷云平面 + 低角度太阳与光晕 + PMREM 环境反射
科技光效克制	6 条低透明度通信弧、扫描环、测绘扇形光束，均无文字与面板
性能与尺度	合批 + InstancedMesh + 关闭实时阴影 + 像素比上限，549 次绘制、78 万三角形

2 运行与操作

2.1 环境与命令

项目使用 Node.js（建议 ≥ 18）与 npm 管理依赖，核心依赖为 `three@0.186` 与 `vite@7`。

Listing 1 安装与运行命令

```
npm install
npm run dev      # 开发服务器 http://127.0.0.1:5179
npm run build    # 生产构建，输出到 dist/
npm run preview  # 预览生产构建
```

2.2 交互方式

场景基于 `OrbitControls` 实现第三人称自由观察，并额外加入“编队跟随”平移：每帧相机与观察目标同时叠加编队质心位移，从而在编队持续巡航时保持构图稳定。

- **左键拖拽**：围绕观察目标旋转，带阻尼；
- **滚轮**：平滑缩放，距离限制 16–4200 m；
- **右键拖拽**：沿水平面平移观察；
- **视角限制**：极角限制 $[0.13, 1.505]$ rad，避免镜头穿入水下；相机始终位于水面以上。

初始镜头位于编队后方偏右的中高空（相对编队坐标系：后退 225 m、上升 72 m、右移 98 m，视场角 47°），俯视约 16°，画面中同时呈现旗舰、前方侦察艇、两翼协同艇、后方保障艇、尾迹与海天交界线。

2.3 开发用取景预设

为便于开发与质量验证，`src/main.js` 内置了一组隐藏的相机预设（不出现在界面中），可通过 `?view=` 查询参数或调试钩子切换，见表 2。

表 2 开发取景预设（不可见于画面）

预设	取景内容	用途
`default`	编队后上方全景，含海天交界线	首屏与整体构图
`low`	水面以上约 5 m，跟随旗舰前进方向	破浪、尾迹、海面低角度
`bow`	旗舰船首左前方约 45 m	船首、球鼻、设备细节
`side`	旗舰右舷侧向近景	船体侧面、灯带、护舷
`fleet`	高高拉起 235 m，回看整个编队	编队秩序与尾迹布局
`top`	编队正上方约 780 m	队形、航迹、海洋纹理
`base`	试验基地、码头与母船	远景设施验证
`reef`	珊瑚礁潟湖俯视	浅水与生态验证



图 2　水面低角度视角：旗舰舷侧、泡沫水线与落日海面。

3　总体架构

3.1　模块划分

代码按“配置—核心—系统—对象”四层组织（图 3）。`config.js` 集中管理调色板、世界布局、编队名册、相机与光照参数，是全部模块共享的单一事实来源；`core/` 提供与具体实体无关的通用能力（几何工具、材质库、波场、海面、天空）；`systems/` 实现编队运动学；`objects/` 按“船—环境—特效”三类组织程序化模型；`main.js` 负责装配、相机、渲染循环与调试钩子。

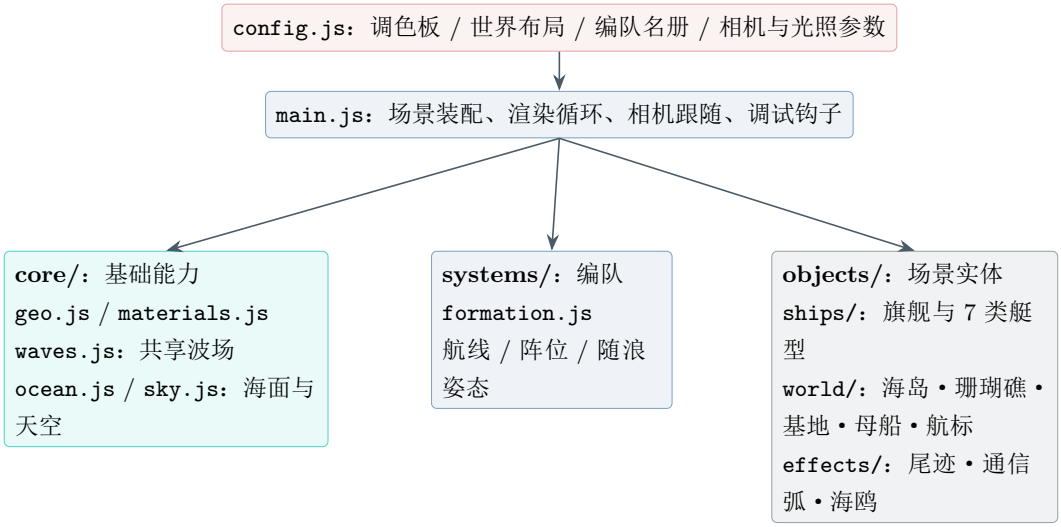


图 3　项目模块架构：配置层、核心能力层、系统层与对象层。

3.2 帧循环与数据流

每帧的计算顺序经过刻意安排，保证“先运动、后接触效果、再环境、最后渲染”（图 4）。特别是尾迹系统需要读取船体最新的世界矩阵，因此必须在 `fleet.update()` 与 `scene.updateMatrixWorld()` 之后更新。

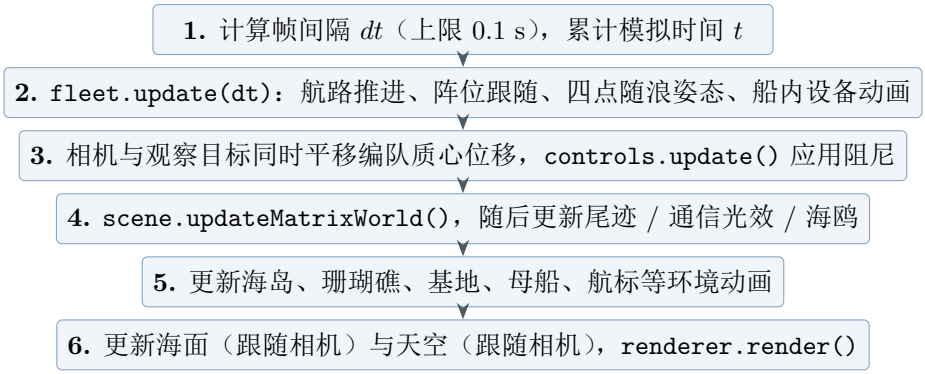


图 4 每帧数据流。

3.3 代码结构与规模

项目共 19 个 JavaScript 源文件，约 13 100 行（不含依赖）。表 3 列出主要文件及其职责。

表 3 主要源文件与职责

文件	行数	职责
`src/config.js`	202	调色板、世界布局、编队名册、相机/光照/雾参数
`src/main.js`	343	渲染器、相机、灯光、模块装配、帧循环、调试钩子
`src/core/geo.js`	594	参数化船体、几何工具集、Builder 合批与描边
`src/core/materials.js`	262	卡通材质库、边缘光、程序化环境反射（PMREM）
`src/core/waves.js`	75	7 组方向波、CPU 采样器、GLSL 波场代码
`src/core/ocean.js`	316	径向海面网格、海洋着色器、浅水与雾
`src/core/sky.js`	378	天空穹顶、积云/卷云、太阳光晕、共享天空色
`src/systems/formation.js`	188	航线、阵位保持、随浪升沉/纵摇/横摇
`src/objects/ships/flagship.js`	1258	30 m 三体旗舰程序化模型与设备动画
`src/objects/ships/fleet-types.js`	1942	7 类无人艇、21 种变体装配
`src/objects/world/islands.js`	1290	海岛高度场、岩壁、植被、拍岸浪
`src/objects/world/reef.js`	1068	珊瑚礁海底、珊瑚、海草、鱼群、焦散
`src/objects/world/base.js`	1468	试验基地、码头、防波堤、建筑群
`src/objects/world/mothership.js`	754	120 m 保障母船
`src/objects/world/markers.js`	898	浮标、灯塔、观测平台、风机
`src/objects/effects/wakes.js`	948	轨迹尾迹、破浪泡沫、尾流
`src/objects/effects/tech.js`	738	通信弧、扫描环、测绘光束
`src/objects/effects/birds.js`	327	海鸥模型与拍翼动画
`src/objects/ships/index.js`	32	按名册组装 17 艘舰艇

3.4 全局约定

- **单位**：1 单位 = 1 米；Y 轴向上；世界坐标原点位于编队初始位置附近。
- **船体局部坐标**：$+X$ 为船首方向，$+Y$ 向上，$+Z$ 为右舷，原点位于水线面船中。`systems/formation.js` 统一负责把局部模型放到世界坐标并施加航向、纵摇、横摇。
- **材质键**：所有静态几何通过字符串材质键引用 `core/materials.js` 中的共享材质，Builder 依据材质键合并网格，从而把每艘船压缩到约 20–25 次绘制调用。
- **动态部件**：雷达、光电云台、航行灯等需要动画的部件通过 `Builder.dynamic()` 单独成组，不参与合批。

4 程序化海洋

海洋是画面中面积最大、最先被感知的元素。项目没有使用任何水面贴图，海面完全由一个自定义 `ShaderMaterial` 生成。

4.1 七组方向波

海面高度由 7 组不同方向、不同波长、不同相速的正弦波叠加而成：

$$y(\mathbf{p},t)=\sum_{i=1}^{7} A_i \sin\!\left(k_i\,\mathbf{d}_i\cdot\mathbf{p}-\omega_i t+\varphi_i\right), \qquad k_i=\frac{2\pi}{\lambda_i}, \tag{1}$$

其中 $\mathbf{p}=(x,z)$ 为水平位置，$\mathbf{d}_i$ 为单位传播方向，A_i 为振幅，λ_i 为波长，φ_i 为初相位。取波长从 132 m 的长涌到 5.6 m 的细浪，形成从大尺度起伏到表面细节的多层次结构。具体参数见表 4。

表 4　程序化波场参数（振幅为米）

序号	传播方向 $\mathbf{d}_i$	波长 λ_i /m	振幅 A_i /m	相速 c_i /(m/s)	初相位 φ_i
1	(0.94, -0.34)	132	0.72	8.8	0.0
2	(0.74, 0.67)	76	0.40	6.4	1.7
3	(0.99, 0.11)	44	0.24	4.8	4.1
4	(0.56, -0.83)	26	0.15	3.6	2.6
5	(0.88, 0.48)	16.5	0.095	2.9	5.3
6	(0.97, -0.24)	9.0	0.052	2.2	0.9
7	(0.72, -0.69)	5.6	0.028	1.8	3.3

4.2 CPU 与 GPU 的一致性

船体必须“看起来真的浮在水面上”，因此 CPU 侧需要实时求任意位置的水面高度与法线。项目把波场定义收敛到 `src/core/waves.js` 一个文件中：同一个 `WAVES` 数组既被 CPU 采样器 `sampleWave()` 使用，也由脚本自动生成 GLSL 常量数组，供海面顶点着色器使用（代码 2）。

Listing 2　共享波场：CPU 采样器与 GLSL 生成

```
export function sampleWave(x, z, t) {
  let y = 0, dx = 0, dz = 0;
  for (const w of WAVES) {
    const ph = w.k * (w.dir[0] * x + w.dir[1] * z) - w.omega * t + w.phase;
    const s = Math.sin(ph), c = Math.cos(ph);
    y  += w.amp * s;
    dx += w.amp * w.k * w.dir[0] * c;
    dz += w.amp * w.k * w.dir[1] * c;
  }
  return { height: y, dhdx: dx, dhdz: dz };   // height + analytic gradient
}

export const WAVE_GLSL = `
  const vec4 WAVE_A[7] = ...;   // generated from WAVES
  const vec2 WAVE_B[7] = ...;
  void sampleWaves(vec2 p, float t, out float h, out vec2 grad) { ... }
`;
```

4.3　海面网格：径向加密与相机跟随

海面网格为半径 26 000 m 的环形网格（236 圈 × 320 段，约 15 万个三角形），径向分布采用“近密远疏”的分段映射：内圈 210 圈按幂函数集中在 2800 m 以内，外圈 26 圈按指数扩展到 26 km。这样相机附近保持约 1–3 m 的顶点间距，可表现短波细节，而远处顶点稀疏但被距离衰减与雾效掩盖。

海面网格每帧水平跟随相机（步长 3 m 吸附以避免抖动），波场始终按世界坐标求值，因此网格移动不会产生“水波跟着镜头走”的穿帮。

顶点着色器中还对每一组波按“波长与相机距离”做衰减（式 2），避免远处稀疏网格对短波采样不足产生锯齿闪烁：

$$w_i(d) = \mathrm{smoothstep}\big(\lambda_i \cdot 24,\ \lambda_i \cdot 150,\ d\big), \qquad h = \sum_i A_i w_i \sin(\cdots), \tag{2}$$

其中 d 为顶点到相机的水平距离。近处（船体所在范围）权重恒为 1，保证船与海的严格一致。

4.4　片元着色

海洋片元着色器（`src/core/ocean.js`）依次处理以下层次：

1. **表面细节法线**：解析波梯度 + 多组高频方向涟漪 + 两组 fbm 细节梯度，仅用于高光与泡沫，避免远景颜色出现噪点；
2. **水体基色**：用“迎光坡度” $\mathbf{N}_{xz} \cdot \mathbf{d}_{sun,xz}$ 决定明暗，得到干净的大尺度斜向光带；再叠加与主涌浪同相的轻微色带、风纹与云影；

3. **浅水潟湖**：4 个椭圆浅滩参数以 uniform 传入，浅水区混入青绿色，并用多尺度 fbm 生成沙洲、深色礁斑与焦散图案；
4. **天空反射**：以水平面反射方向与波浪反射方向混合采样 `skyTint()`，乘以菲涅尔系数；
5. **太阳镜面与碎金**：Blinn-Phong 高光（宽高光 + 尖锐闪烁）叠加噪声 glitter，形成从地平线延伸至中景的金色反射带；
6. **泡沫**：仅在高波峰与浅滩边缘生成稀疏白浪，避免整片海面出现噪点；
7. **大气雾**：按指数雾将水面颜色逐方向混合到天空地平线颜色，确保海天无缝；
8. **透明度**：深水基本不透明，浅水透明度提高（alpha ≈ 0.72），使海底沙脊、珊瑚与鱼群透出水面。

Listing 3　海面片元着色器的核心水体与反射计算（GLSL 摘录）

```
vec3 Ncolor = normalize(vNormal);                // smooth vertex normal
float lightSide = smoothstep(-0.28, 0.30, dot(Ncolor.xz, uSunDir.xz));
vec3 base = mix(uDeepColor, uMidColor, 0.34 + 0.44 * lightSide);
base = mix(base, uLightColor, smoothstep(0.62, 1.0, lightSide) * 0.30);
base *= 0.90 + 0.18 * sunDiff;

vec3 Rflat = reflect(-V, vec3(0.0, 1.0, 0.0));
vec3 Rwave = reflect(-V, N);
vec3 skyRef = mix(skyTint(normalize(Rflat)), skyTint(normalize(Rwave)), 0.16) * 0.94;
float fres = 0.018 + 0.978 * pow(1.0 - clamp(V.y, 0.0, 1.0), 4.5);
col = mix(base, skyRef, clamp(fres * 0.78, 0.0, 0.62));

// distance-dependent exponential fog to the sky horizon colour
vec3 horizonCol = skyTint(normalize(vec3(hdir.x, 0.0, hdir.y)));
col = mix(col, horizonCol, 1.0 - exp(-pow(camDist * uFogDensity, 2.0)));
```

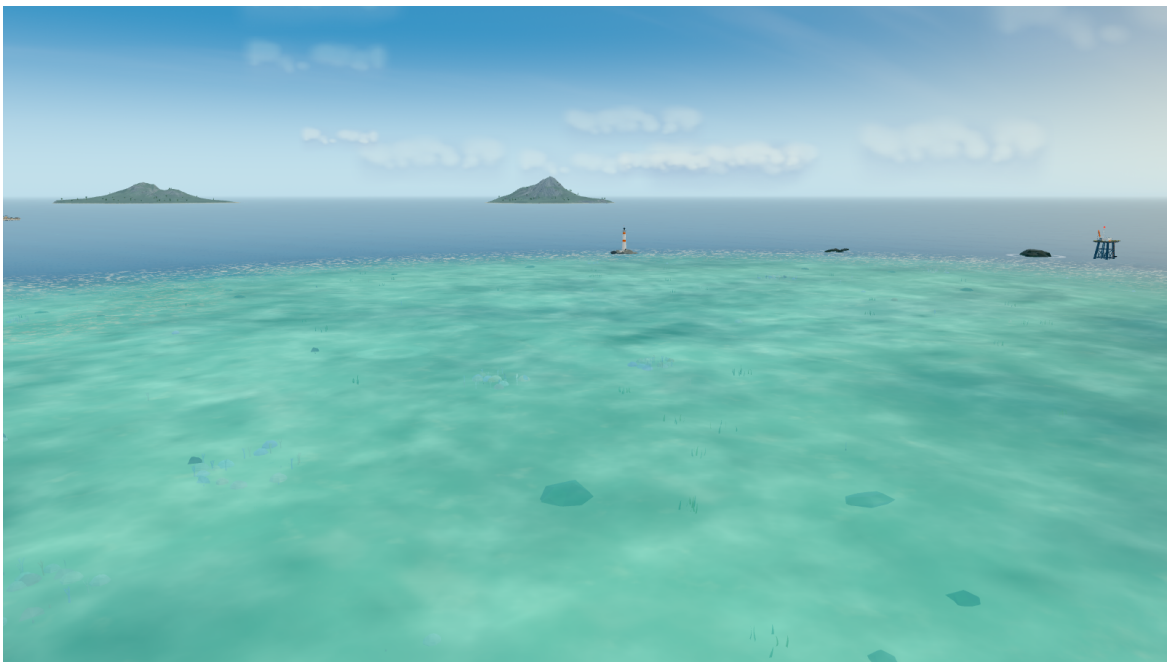


图 5　浅海珊瑚礁潟湖：海面半透明，海底沙脊、暗色礁斑、珊瑚与海草透水可见。

5　天空、大气与光照

5.1　天空穹顶

天空使用一个跟随相机的巨大穹顶（半径 48 km）与专用 ShaderMaterial 绘制。片元着色器沿视线方向计算：

- **垂直渐变**：地平线（`0xcae6ef`）→ 中层天空（`0x4f9dca`）→ 天顶深蓝（`0x1c5a96`）；
- **太阳散射**：围绕太阳方向的宽散射与暖色调混合，太阳本体为高亮圆盘，叠加三层不同指数的高光晕；
- **高空卷云**：用方向投影坐标上的 fbm 噪声绘制极淡的丝缕状卷云；
- **地平线雾带**：$e^{-26|h|}$ 的浅色带，使地平线柔和自然；
- **抖动**：加入极小的高频抖动，消除 8-bit 渐变条带。

穹顶材质关闭深度测试与深度写入，并设置 `renderOrder = -1000`，保证它永远在所有不透明物体之后、其他半透明物体之前绘制。

5.2　海天无缝：共享天空色函数

海洋着色器与天空穹顶共用同一段 GLSL ——`skyTint(vec3 dir)`。海面的远处雾色不是固定颜色，而是按视线水平方向查询天空颜色（图 6）。因此：

- 天与海的交界线处，两边颜色逐像素一致，不会出现“白色地平线带”或“海水断层”；

- 朝向太阳一侧的地平线自动带暖金色，背光侧偏冷蓝，海面反射与雾色也随之变化；
- 整片海面不需要额外的“地平线雾墙”或背景板。



图 6　低角度侧视：天空渐变、太阳光晕、海天交界线与船体边缘光。

5.3　云与太阳

云层分为三层：

- **积云**：26 个公告板精灵（Sprite），使用 3 张程序化画布纹理（1024×512）。纹理由数十个柔性径向渐变“云团”堆叠，上半部为暖白受光面，下半部为蓝灰背光面，并在画布边缘做羽化，避免出现矩形裁切。云精灵分布在水平 5–14 km、仰角约 1–10° 的范围内，随距离逐渐偏向雾色；
- **高空卷云**：5 张水平放置的半透明条带平面（约 9–18 km 宽），缓慢旋转漂移；
- **太阳光晕**：一个加法混合的径向渐变精灵，位于太阳方向 3×10^4 m 处，克制地提亮太阳周围天空。

5.4　光照与材质响应

场景采用“暖色低角度主光 + 冷色半球环境光 + 冷色补光”的三灯方案（表 5）。实时阴影被关闭：一方面海面使用自定义 ShaderMaterial，接入阴影贴图的成本与复杂度较高；另一方面三渲二风格通过卡通阶跃光照、边缘光与假接触泡沫已经足以表达体积。船体与水面之间通过尾迹泡沫、船首破浪与水线压暗来建立接触关系。

表 5　光照配置

光源	颜色	强度	方向/作用
平行光（太阳）	`0xffeccb` 暖白	1.55	仰角约 17.5°，船体亮面与海面碎金
半球光	天 `0xd2e6f2` / 地 `0x2a5a6c`	1.05	填充背光面，形成蓝灰阴影
平行光（补光）	`0xbcd6ee` 冷蓝	0.55	从太阳反方向补冷光，增强轮廓
环境反射	程序化天空立方图	—	金属、玻璃的 PMREM 反射

6　无人艇程序化建模

6.1　统一设计语言与尺度

全部 17 艘无人艇遵循统一工业设计语言：修长船体、低矮封闭设备舱、模块化任务载荷、深色玻璃传感器窗口、珍珠白 + 海军灰 + 深海军蓝涂装，少量信号橙与荧光青作为识别色。尺寸覆盖 10.5–30 m，旗舰作为尺度参照（表 6）。

表 6　无人艇类别与主要参数

类别	数量	长度/m	型宽/m	吃水/m	主要特征
旗舰	1	30.0	9.4	1.7	三体、双雷达罩、通信桅杆、旋转雷达、光电球、UUV 机库、折臂吊
侦察艇	5	10.5	3.0	0.62	深 V 高速艇体、喷水推进、球形光电、扫描雷达、橙/青识别区
环境监测	3	16.5	5.6	1.0	实验室舱、尾部自动采样平台、升降机构、采样瓶架、折叠探针
海洋测绘	2	18.0	7.4	1.1	对称双体、中央传感器通道、声呐吊舱、A 形门架、双 GNSS 罩
多用途巡逻	2	14.5	4.7	0.9	模块化任务集装箱、雷达、光电、双天线
通信中继	1	19.0	6.2	1.15	11 m 桁架桅杆、多组微波天线、卫星罩、索槽
能源保障	2	20.0	7.0	1.2	电池柜阵列、太阳能板、甲板吊、充电卷盘、维护平台
无人救援	1	15.5	5.4	1.0	橙白涂装、救援平台、折臂、救生圈发射架、担架

6.2　参数化船体生成

每条船体由 `core/geo.js` 中的 `hullParts(spec)` 生成。其核心是把船体看作沿长度方向变化的横截面族：对参数 $u \in [0,1]$（船尾到船首）定义半宽、吃水、干舷、外飘、艏柱前倾与截面幂次，再用密排截面放样成网格（代码 4）。

Listing 4　参数化船体截面与船体部件拆分

```
// spec: length, beam, freeboard, sheer, draft, bowSharp, flare,
//       rocker, sternWidth, camber, sectionPower, rake
```

```
const parts = hullParts({
  length: 30, beam: 5.6, freeboard: 1.46, sheer: 0.74,
  draft: 1.635, bowSharp: 1.16, flare: 0.3, rocker: 0.55,
  sectionPower: 1.72, rake: 0.24, uSegments: 56,
});
// parts = { lower, topsides, deck, transom }
// lower    -> keel to waterline (deep navy antifouling)
// topsides -> waterline to gunwale (pearl white)
// deck     -> cambered deck cap
// transom  -> flat stern panel
```

上下两部分独立生成、共享水线顶点，因此水线是一条干净锐利的色阶分界，而不是插值渐变；再叠加 `hullBand()` 生成的深蓝腰线、金属护舷条与青色识别灯带。该方法同时服务于单体艇、双体测绘艇的两片船体与三体旗舰的中央主船体/两个浮体。

6.3 Builder：合批、描边与动态部件

逐零件建模会产生成千上万个 Mesh。`core/geo.js` 的 `Builder` 负责将静态几何按材质键合并为不超过 10 余个网格，并额外生成一个“反向外壳”描边网格（代码 5）。

Listing 5 Builder 的典型用法

```
const b = new Builder(materials, { outlineMaterial: materials.outlineBoat });
b.solid(parts.lower,  'hullBottom', { pos });   // merged + outline
b.solid(parts.topsides,'pearl',    { pos });
b.add(railing,        'metal',     { pos: [...] });  // merge only
const radar = b.dynamic('radar');                // animated sub-group
radar.add(new THREE.Mesh(radarBarGeo, materials.metalDark));
const group = b.build();                         // 1 draw per material + outline
```

描边材质为背面材质，顶点着色器中沿法线向外扩张固定宽度（约 0.13 m），形成干净的三渲二轮廓线，同时不产生额外的屏幕空间后处理开销。

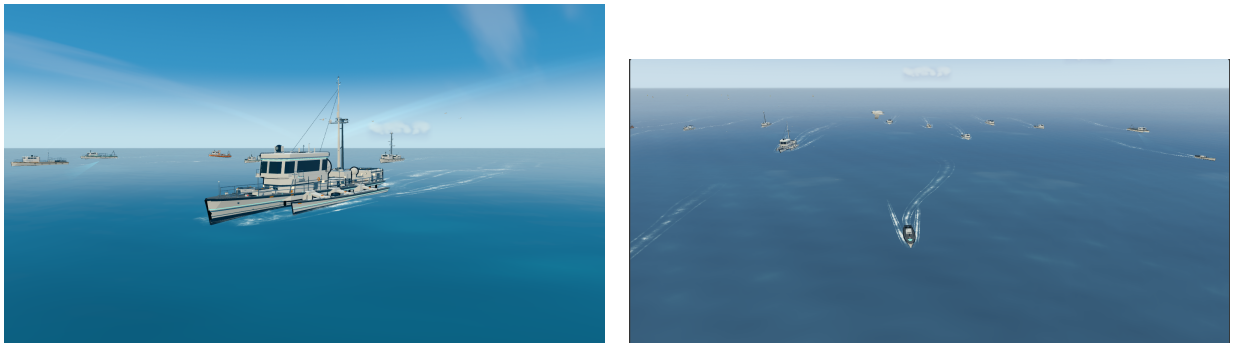
6.4 旗舰设计

旗舰是本场景的核心视觉焦点，采用三体结构：中央主船体长 30 m，两侧各一个 20 m 浮体（间距 3.94 m），通过三组后掠横臂与斜撑连接；船体为尖锐破浪艏、低重心、宽阔中央任务甲板与流线型外壳。主要构成：

- **上层建筑**：前部后掠式驾驶/设备舱，环绕式深色玻璃带，珍珠白折面舱顶，青色灯带与暖色工作窗；
- **感知设备**：大型白色卫通雷达罩（半径 1.1 m）、后部小型雷达罩、8 m 通信桅杆、多组刀形天线、旋转式海事雷达、前部可转光电球、圆柱激光雷达、气象站；
- **任务设备**：后部 UUV 机库（带凹陷式艉门与橙色警示条纹）、折叠关节吊臂、模块化任务舱口、甲板系缆桩与检修舱盖；

- **细节**：黑色橡胶分段护舷、嵌入式金属护栏、排水孔、冷却格栅、面板接缝、航行灯（左红右绿）、桅顶白灯、青色识别灯带；
- **动态部件**：旋转雷达、缓慢扫描的光电云台、周期闪烁的桅顶警示灯。

旗舰共 61 302 个三角形（其中描边约 12.6k），13 个静态材质批次 + 1 个描边 + 3 组动态网格。



(a) 旗舰船艏方向　　(b) 编队上空近景

图 7　旗舰与集群实机截图。

6.5　材质库与三渲二外观

`core/materials.js` 集中定义材质：

- **卡通材质**：`MeshToonMaterial` + 4 阶梯度贴图，并在 `onBeforeCompile` 中注入菲涅尔边缘光（rim），使背光面仍保持清晰轮廓；
- **金属/玻璃**：`MeshStandardMaterial` + 程序化 PMREM 环境反射，分别控制金属度与粗糙度；
- **发光件**：航行灯、识别灯带、桅灯使用无光照的 Basic 材质；
- **描边**：背面材质 + 顶点法线外扩。

7　编队动力学与航行

7.1　航线

编队沿一条极宽的左转圆弧前进。设航向角 $\theta(t)$ 随时间线性增加，则旗舰轨迹可由积分得到解析式：

$$\begin{aligned} x(t) &= x_0 + \frac{v}{\omega}\big(\sin\theta(t) - \sin\theta_0\big), \\ z(t) &= z_0 + \frac{v}{\omega}\big(\cos\theta(t) - \cos\theta_0\big), \end{aligned} \qquad \theta(t) = \theta_0 + \omega t, \tag{3}$$

其中 $v = 3.1$ m/s，$\omega = 0.0019$ rad/s，转弯半径约 1.63 km。该半径使编队在数分钟内只缓慢改变航向，形成“宽阔弧形转弯”的视觉效果，同时不会让相机跟随产生突兀旋转。

7.2 阵位与自然摆动

17 艘舰艇在“编队坐标系”（前向 +、右向 +）中拥有固定阵位：旗舰位于原点；5 艘侦察艇在正前方 46–186 m 呈箭头展开；两侧中型艇在 −268 m 至 +224 m 的横向范围内错列；4 艘保障艇位于后方 118–134 m。阵位之上叠加多组低频正弦摆动，使编队保持严谨秩序的同时具有自然呼吸感；侦察艇还额外叠加周期性横向展开项，模拟“向外搜索—回归编队”的行为。

7.3 跟随与航向控制

每艘艇用指数弹簧逼近目标阵位（式 4），并用“航路方向 + 位置修正”的混合向量计算期望航向，再对航向做角度平滑，从而在转弯时自然出现内切与外扩：

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \left(\mathbf{p}^* - \mathbf{p}_k\right)\left(1 - e^{-dt/\tau}\right), \qquad \psi^* = \arg\Big(\mathbf{d}_{course} + \alpha(\mathbf{p}^* - \mathbf{p})\Big). \tag{4}$$

不同吨位的跟随时间常数 τ 与转向响应不同，使小艇更灵活、保障艇更沉稳。转弯时根据角速度给高速艇叠加轻微侧倾（bank）。

7.4 随浪姿态

船体姿态不使用解析波梯度直接计算，而是对船体“首、尾、左、右”四个点采样水面高度，再按船长与型宽求纵摇与横摇（式 5）。这样 30 m 的旗舰不会被 5 m 的细浪直接抬起，姿态被船体尺度自然低通，同时小船仍然灵活。

$$\begin{aligned} \text{pitch}^* &= \arctan\frac{h_{bow} - h_{stern}}{2L_h}\,\gamma_p + \text{planing}, \\ \text{roll}^* &= -\arctan\frac{h_{right} - h_{left}}{2B_h}\,\gamma_r + \text{bank}, \\ h^*_{heave} &= 0.35\,h_c + 0.1625\,(h_b + h_s + h_r + h_l), \end{aligned} \tag{5}$$

其中 γ_p, γ_r 为按艇型配置的响应系数（高速艇接近 1，双体测绘艇约 0.4）。升沉、纵摇、横摇均经过指数平滑，避免数值抖动。



图 8　高空俯视：17 艘舰艇的编队秩序、航向与 17 条白色尾迹。

8　尾迹、泡沫与协同光效

8.1　历史轨迹尾迹

每艘船保存两条世界坐标历史缓冲：近端 32 个采样点（每 0.1 s 一个，约 3.2 s）与稀疏长尾 128 个点（每 0.5 s 一个，最长约 55 s）。每帧根据最近采样点重建一条包含 46 个横截面的尾迹带：

- 每个横截面根据船体航向计算左右方向，宽度随“年龄”从约 $0.60 \times W_{wake}$ 扩展到 2.4–$3.1 \times W_{wake}$；
- 顶点高度实时采样共享波场，使尾迹贴合起伏的海面而不是悬空平面；
- 每条尾迹约 90–160 个三角形，使用动态缓冲原地更新，不产生每帧临时对象。

8.2　泡沫着色器

尾迹使用统一的自定义 ShaderMaterial，按几何类型分为：中央尾迹带、船首两条 V 形扩散臂、船体两侧水线泡沫线、船首泡沫项圈、尾部推进泡沫与扰动区。片元着色器用三种噪声叠加出世界坐标系的泡沫场：

- 泡沫沿航行方向缓慢流动，并随“年龄”衰减透明度；
- 中央亮带 + 两条边缘细带形成破浪结构，船首 V 臂额外增亮；
- 大尺度斑块与小尺度空洞让泡沫呈“被搅动的含气水”而不是一块白色平面；
- 冷色泡沫阴影与暖色受光面混合，最后按距离做大气衰减。

8.3 协同可视化光效

为表达“集群智能”，场景加入极克制的科技光效（全部不使用文字、数字与面板）：

- **通信弧**：6 条低透明度加法混合弧线，在旗舰与协同艇之间短距离连接，光脉冲沿弧线行进，周期 38–62 s、出现时长 12–18 s、透明度仅 0.05–0.09；
- **扫描环**：侦察艇前方偶尔出现的水面扩散圆环，低透明度、短暂；
- **测绘扇形**：测绘艇下方极淡的水下扇形探测光束，周期脉冲。

8.4 海鸥

8 只程序化低模海鸥在编队附近 8–40 m 高度盘旋，翅膀按“拍动—滑翔”节奏摆动，身体随转弯轻微侧倾，为静态的工业场景加入生命感。

9 海域环境构建

9.1 海岛地形

海岛由噪声位移的径向高度场生成，而非圆锥或球体拉伸：

- **近处研究岛**：半径 300 m、最高 92 m。面向船队一侧抬升为分层岩壁与海蚀崖，岛内为起伏绿色丘陵；基地区域被刻出一块约 10–12 m 高的低平海岸台地，供工程建设使用；
- **植被**：两套低模树（细树干 + 多层树冠）以 InstancedMesh 种植 472 棵，避开基地施工区；
- **海岸**：56 块岸石、48 处岩层/海蚀柱、3 条缓慢脉动的拍岸浪环，构成自然的岸线节奏；
- **两座中景岛**（高 220/170 m）：非对称山脊、裸岩带与海蚀柱，颜色随距离变冷；
- **远景岛**（高 430 m）：低模轮廓剪影，在雾气中呈蓝灰色。

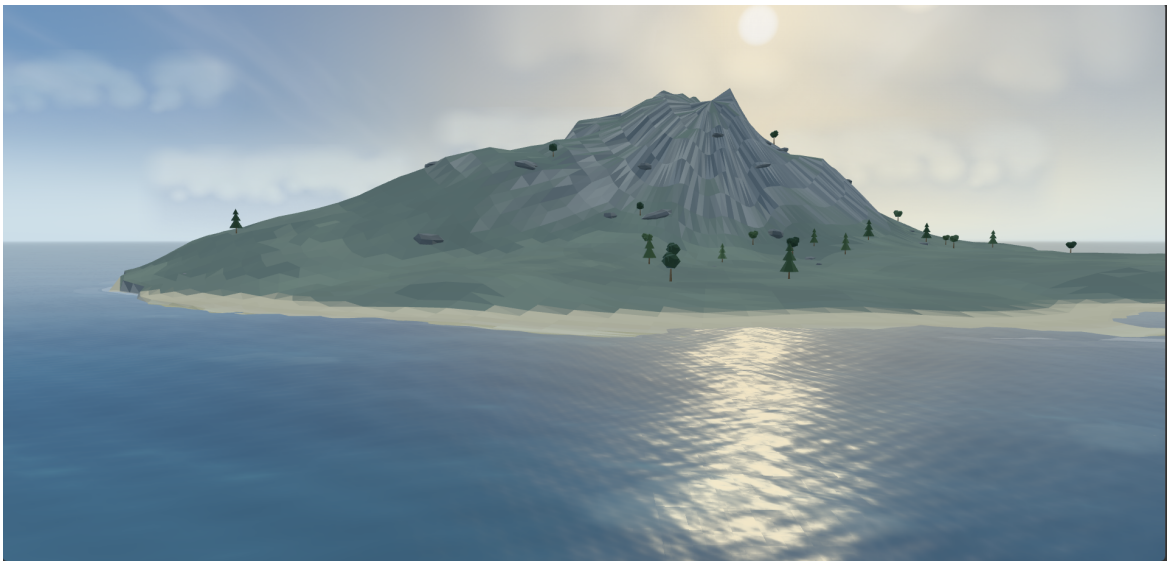


图 9　中景海岛近观：低多边形丘陵、裸岩山脊、树木与浅滩沙线。

9.2 珊瑚礁潟湖

右侧中景是一片直径约 1.5 km 的浅海珊瑚礁。它由四个椭圆浅滩场驱动，海底为由 224×80 网格构成的平滑沙地（约 4.9 万三角形），叠加：

- 498 组珊瑚：分枝状、扇形、脑状/海绵状三类模板，以 InstancedMesh 实例化，色彩为粉、橙、紫三组，簇拥成花园并留出沙质通道；
- 661 片随顶点着色器摆动的大型海草；
- 52 块水下碎石、7 处露出水面的礁石与两座小礁岛（带泡沫环）；
- 3 个鱼群共 136 条小鱼，在珊瑚上方缓慢环绕；
- 8 个缓慢漂移的焦散光环。

海面在这些浅水区半透明，海底与珊瑚从水面下透出，与无人艇的环境监测/测绘任务形成叙事联系。

9.3 海上试验基地

基地位于近岛面向船队一侧，由大型混凝土/岩石平台、防波堤、三组桩基码头与建筑群构成：

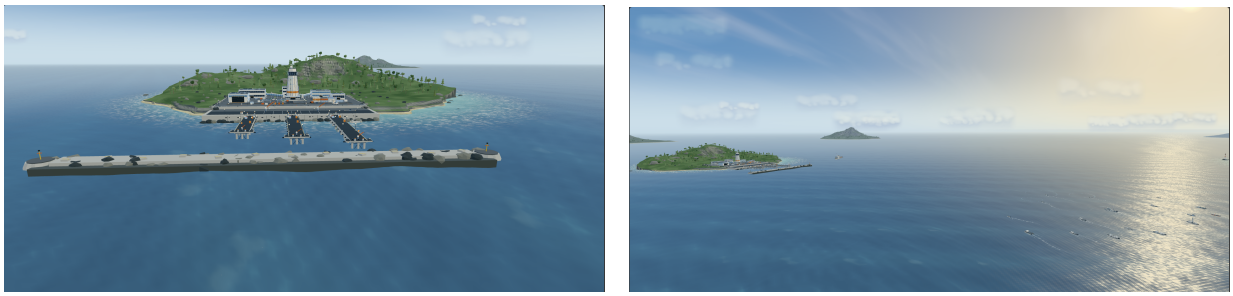
- **指挥塔**：约 62 m 高、倾斜切面的塔体，上部深蓝色玻璃观测带、环形外平台、白色雷达罩、旋转雷达与红色航空警示灯；
- **建筑群**：5 栋模块化科研/办公建筑，银白立面、深色玻璃带、屋顶太阳能板与设备；3 条封闭连廊连接；
- **码头**：三组向海伸出的桩基码头（100/125/145 m），防滑甲板、系泊桩、黑色护舷、灯柱、吊机、设备箱与停泊的无人艇；
- **其他**：集装箱堆场、油料/能源模块、太阳能棚、防波堤与灯标。建筑与地形通过地形高度采样函数贴合，不留悬空缝隙。



图 10　海上无人系统试验基地：指挥塔、模块建筑、桩基码头、防波堤与停泊艇。

9.4　母船与航标设施

- **保障母船**：120 m 级海洋工程船，浅灰白船体、深蓝水线、开放式尾部任务甲板、门式吊机、无人艇停放位、集装箱与通信桅杆，系泊于深水区，缓慢升沉；
- **导航浮标**：18 个智能浮标（橙白涂装、太阳能板、雷达反射器、闪烁信号灯）与黄色航道标、白色科研浮标，全部按共享波场随浪起伏；
- **灯塔**：建在礁石小岛上，约 28 m 白色塔身、观景平台、灯室与暖白旋转灯光；
- **观测平台**：钢制导管架腿 + 甲板 + 设备舱 + 通信桅杆 + 直升机坪的小型自升式平台；
- **海上风电场**：5 台白色风机，塔高约 90–100 m，叶片以不同速率缓慢旋转。



(a) 从海面看基地与防波堤　　(b) 广角：海岛、基地、编队与海天

图 11　环境实机截图。

10 美术方向、色彩与构图

10.1 三渲二的混合管线

场景并非全部使用卡通材质，而是按物体类型混合多种表现：

表 7 不同对象的表现方案

对象	渲染方式	目的
船体/建筑/地形	卡通材质 + 4 阶梯度 + 边缘光	干净的色块、清晰的轮廓与体积
金属/玻璃	Standard + PMREM 天空反射	工业精密感与真实反射
海面/天空	自定义 ShaderMaterial	程序化波浪、菲涅尔、碎金、无缝地平线
尾迹/光效	半透明自定义着色器	泡沫流动、通信光脉冲
描边	反向外壳	三渲二轮廓，无后处理依赖

10.2 主色板

表 8 为主要色彩及其在场景中的职责。整体以深海蓝、湛蓝、青绿、海军灰、珍珠白与科技银为主，信号橙与荧光青作为少量识别色。

表 8 主色板

色块	名称	HEX	用途
	珍珠白 pearl	F4F6F4	船体亮面、雷达罩、建筑外墙
	科技银 silver	C5D0D6	栏杆、桅杆、金属结构
	深海军蓝 navy	173A56	水线带、玻璃、艇底
	深海 hullDeep	0D5F80	海洋基色（深水）
	浅海 shallow	3ECFBD	珊瑚礁浅水
	浪尖 crest	C8F5E8	波峰高光、泡沫提亮
	荧光青 techCyan	4CEFE0	灯带、通信弧、扫描环
	信号橙 signal	FF8C2E	安全标记、救援艇、浮标
	暖金 sun	FFD27A	太阳、碎金反射、暖色灯
	岩灰 rock	7A8488	海岛岩壁、礁石
	海雾 haze	B7D6E5	大气雾、远景空气透视

10.3 构图与镜头语言

初始镜头（图 1）刻意采用“斜向引导线 + 三层景深”的构图：

- **前景**：近处海面波纹、旗舰与保障艇的船体、白色尾迹，占画面下半部；
- **中景**：完整的 17 艇协同编队，沿右上方延伸，形成斜向视觉引导；
- **远景**：左侧海岛与基地、中央远山、右前方珊瑚礁、地平线云层，形成多层次的剪影；
- **天空带**：画面上方保留约 10–15% 的天空与海天交界线，暖阳位于右前方地平线附近，在海面拉出碎金反射带。

相机视场角 47°、俯仰约 16°，在当前距离下旗舰约占屏高 8–10%，既可辨认艇型，又能容纳完整编队。用户可缩放至 16 m 观察船体细节，或拉高至 4.2 km 俯瞰整个海域。

11 性能优化与质量验证

11.1 性能预算与优化手段

- **几何合批**：静态几何按材质合并，每艘船约 20–25 次绘制；16 艘量产艇总计约 38.5 万三角形、约 385 次绘制（含描边与动态件），旗舰约 6.1 万三角形、14 次静态绘制；
- **实例化**：树木、珊瑚、海草、鱼群、浮标、风机叶片等重复元素使用 InstancedMesh；
- **共享材质**：统一材质库避免重复编译着色器；材质键驱动合批；
- **波场层级**：对远处按波长衰减波组，避免细浪采样不足；
- **无实时阴影**：关闭 shadow map，改为尾迹泡沫、水线压暗与卡通边缘光表达体积；
- **像素比上限**：`devicePixelRatio` 上限 1.75，平衡高分屏清晰度与填充率；
- **动态缓冲复用**：尾迹、海鸥、鱼群等每帧原地更新预分配缓冲，不在循环中分配对象。

图 12 给出各模块三角形预算的直观对比（单位：千三角形，包含合批后的实际提交量）。

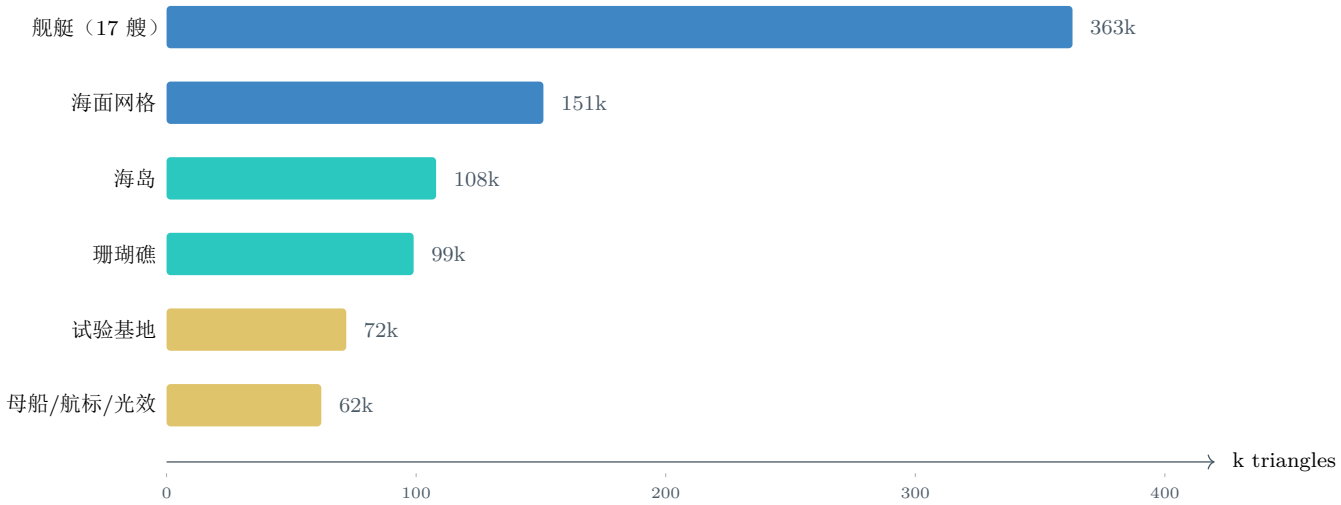


图 12　各模块三角形预算（约值，单位：千三角形）。

11.2 实测数据

在无头浏览器（Chromium + SwiftShader 软件渲染，1600×900）下通过 Playwright 自动化测试，得到表 9 的数据。软件渲染的帧率不代表真实 GPU 表现，但绘制调用、三角形数量与控制台状态具有参考意义。

表 9　实测渲染与质量指标

指标	数值	说明
绘制调用	约 549	单帧 renderer.info.render.calls
三角形数量	约 78.4 万	单帧提交
着色器程序	约 50	共享材质显著减少程序数
无人艇数量	17	1 旗舰 + 16 艘任务艇
控制台错误	0	含页面异常、资源失败、着色器错误
界面文本节点	0	DOM 审计：1 个全屏 canvas，无覆盖层
生产构建体积	859 KB（gzip 242 KB）	Vite 生产构建

11.3　自动化视觉验证

项目在开发过程中使用 Playwright + 无头 Chromium 构建了一套自动化验证流程：

1. 启动开发服务器并等待 `window.__oceanSwarm.ready === true`；
2. 收集控制台错误、页面异常、未处理 Promise 与失败请求；
3. 通过隐藏的 `setView(name)` 钩子切换 8 个取景预设并分别截图；
4. 暂停渲染循环后截图，规避软件渲染下的截图超时；
5. 对截图做黑屏/空白检测、绘制调用与三角形统计、DOM 覆盖层审计；
6. 人工复核截图，按 P0/P1/P2 对视觉问题分级并迭代修复。

最终一轮验证结果：零控制台错误、零页面异常、零失败请求，画面中不存在任何 UI 文本或面板；海洋连续、地平线完整、17 艘舰艇全部出现、尾迹方向与航向一致、岛屿与基地尺度合理。

11.4　已知限制与后续方向

- **无实时阴影与后处理**：目前依靠卡通光照与泡沫建立体积感，后续可加入低成本屏幕空间环境光遮蔽、软阴影或辉光；
- **水面为解析波叠加**：没有 FFT 频谱与真实的波浪破碎模拟，尾迹泡沫为程序化近似；
- **头部帧率不具代表性**：无头 SwiftShader 只有 2–3 fps，真实 GPU 上预期 30–60 fps；
- **移动端未验证**：高像素比与大量三角形可能对移动 GPU 有压力，可按设备分档降低海面网格与阴影质量；
- **可扩展方向**：WebGPU / 计算着色器 FFT 海面、动态天气与昼夜循环、任务级编队行为（避障、搜索、回收）、视频录制与在线发布。

12 总结

本文档记录了一个从零开始、完全程序化的未来海洋无人系统三维场景。项目以 Three.js 与 Vite 为基础，在无任何外部美术资源的约束下，通过参数化船体生成、共享波场、方向相关天空色、历史轨迹尾迹与分层环境构建，实现了数公里尺度的连续海洋、17 艘协同巡航的智能无人艇编队，以及由海岛、基地、珊瑚礁、航标、风机与云层共同构成的完整海天世界。

在工程上，项目通过材质键合批、实例化、动态缓冲复用、关闭实时阴影与像素比上限等手段，将场景控制在约 549 次绘制调用与 78 万三角形；通过自动化截图与 DOM/控制台审计，确认了零错误、零 UI 的验收结果。完整源码、构建方式与全部实机截图均可在项目仓库中获取：

https://github.com/ZHT1235456/OceanSwarm

A 波场采样与姿态计算补充

由式 (1) 可得海面高度对水平坐标的解析梯度：

$$\frac{\partial y}{\partial x} = \sum_i A_i k_i d_{i,x} \cos(\cdots), \qquad \frac{\partial y}{\partial z} = \sum_i A_i k_i d_{i,z} \cos(\cdots), \tag{6}$$

表面法线为

$$\mathbf{n} = \frac{1}{\sqrt{1 + |\nabla y|^2}} \left(-\frac{\partial y}{\partial x},\ 1,\ -\frac{\partial y}{\partial z} \right). \tag{7}$$

CPU 采样器返回高度与梯度，供船体四点姿态采样与水下物体随浪起伏使用；GPU 顶点着色器在相同公式上叠加距离衰减权重，保证船舶附近的水面与物理采样严格一致。

B 项目目录结构

Listing 6 项目目录结构

```
OceanSwarm/
  index.html                # entry page (full-screen canvas only)
  package.json / vite.config.js / README.md
  docs/                     # this document, PDF and 12 figures
  src/main.js               # scene assembly and render loop
  src/config.js             # palette / layout / formation / lighting
  src/core/                 # geometry, materials, waves, ocean, sky
  src/systems/              # formation kinematics
  src/objects/              # ships / world / effects
```